

Competitive Programming

Section 44: String Algorithms II

Competitive Programming Course

2026-07-06

Table of contents

1	Tries, palindrome checks, and suffix automata	3
1.1	Trie prefix counting	3
1.2	Palindrome substring queries	3
1.3	Distinct substrings and suffix automaton idea	4
1.4	Recognition table	4
1.5	Distinct substrings by hand	4
1.6	Trie example	5
1.7	Hash versus automaton versus trie	5
1.8	Practice drill	5
2	Exercises	6

1 Tries, palindrome checks, and suffix automata

A trie stores strings by shared prefixes and makes prefix counting a graph walk. Palindrome queries can be checked with hashes in large settings; the local exercise first uses direct mirrored comparisons so the equality condition is completely visible before introducing hashes. A suffix automaton compactly represents all substrings: each state contributes `len[state] - len[link[state]]` new substrings.

1.1 Trie prefix counting

A trie is a rooted tree where each edge is labeled by a character. A word is a root-to-node path. To answer “how many dictionary words have prefix `p`”, store `cnt[v]`: how many inserted words pass through node `v`.

Insertion:

```
node = root
for c in word:
    create edge c if missing
    node = child[node][c]
    cnt[node] += 1
```

Query:

```
walk the prefix
if an edge is missing: answer 0
else answer cnt[last_node]
```

The invariant is simple: after all insertions, `cnt[v]` equals the number of words whose path includes `v`.

1.2 Palindrome substring queries

For a substring `s[l..r]`, the direct check compares mirrored characters:

```
s[l] with s[r]
s[l+1] with s[r-1]
...
```

This is $O(\text{length})$ and is useful for teaching correctness. In larger contests, use rolling hashes:

```
hash_forward(l,r) == hash_reverse(n-1-r, n-1-l)
```

Hashing is fast but probabilistic unless you use deterministic structures such as suffix arrays or palindromic trees. When constraints permit direct checking, the exact method is safer.

1.3 Distinct substrings and suffix automaton idea

A suffix automaton is a compressed DFA containing all substrings of a string. For each state v :

- $\text{len}[v]$ is the maximum length of a substring represented by v ;
- $\text{link}[v]$ points to the state representing the longest proper suffix class.

The number of new substrings contributed by v is:

```
len[v] - len[link[v]]
```

Summing this over all states except the initial state gives the number of distinct non-empty substrings.

For a pedagogical local problem, suffix-array or direct-set solutions may be used at small limits, but the formula above is the red-level invariant you should know.

1.4 Recognition table

Problem form	Tool
dictionary prefixes	trie
repeated substring equality	rolling hash or suffix array
palindrome range query	forward/reverse hash, Manacher, eertree
count distinct substrings	suffix automaton or suffix array

1.5 Distinct substrings by hand

For $s = \text{aba}$, the distinct non-empty substrings are:

```
a, b, ab, ba, aba
```

There are 5. A suffix automaton reaches the same count by summing:

```
len[state] - len[link[state]]
```

over states. The formula counts how many new substring lengths become possible when that state is created. The direct enumeration is fine for tiny strings; the automaton is what makes the same idea linear for large strings.

1.6 Trie example

Insert:

```
app, apple, ape
```

The node for prefix **ap** has counter 3. The node for **app** has counter 2. The node for **apple** has counter 1. A missing edge answers 0 immediately.

1.7 Hash versus automaton versus trie

These tools solve different string questions:

```
trie: many dictionary prefixes  
hash: fast equality checks between substrings  
suffix automaton: all substrings of one string
```

Do not use a suffix automaton for prefix dictionary queries; it is the wrong shape. Do not use a trie to compare arbitrary substrings inside one long text. Choosing the representation is the main skill in this section.

1.8 Practice drill

For any string problem, write which object is being queried:

```
prefix of dictionary word  
substring of one text  
suffix of one text  
palindrome centered somewhere  
set of all substrings
```

Then choose the representation. A trie is a prefix tree. A hash is a comparison tool. A suffix automaton is a compressed representation of all substrings. This one sentence prevents many overcomplicated solutions.

2 Exercises

- A. Trie Prefix Count: Given dictionary words and prefix queries, count words having each prefix.
- B. Palindrome Queries: For each substring query, print whether the substring is a palindrome.
- C. Distinct Substrings: Print the number of distinct non-empty substrings of a string.